

UDP多播（组播）、广播与domain进程间通信

UDP局域网服务器

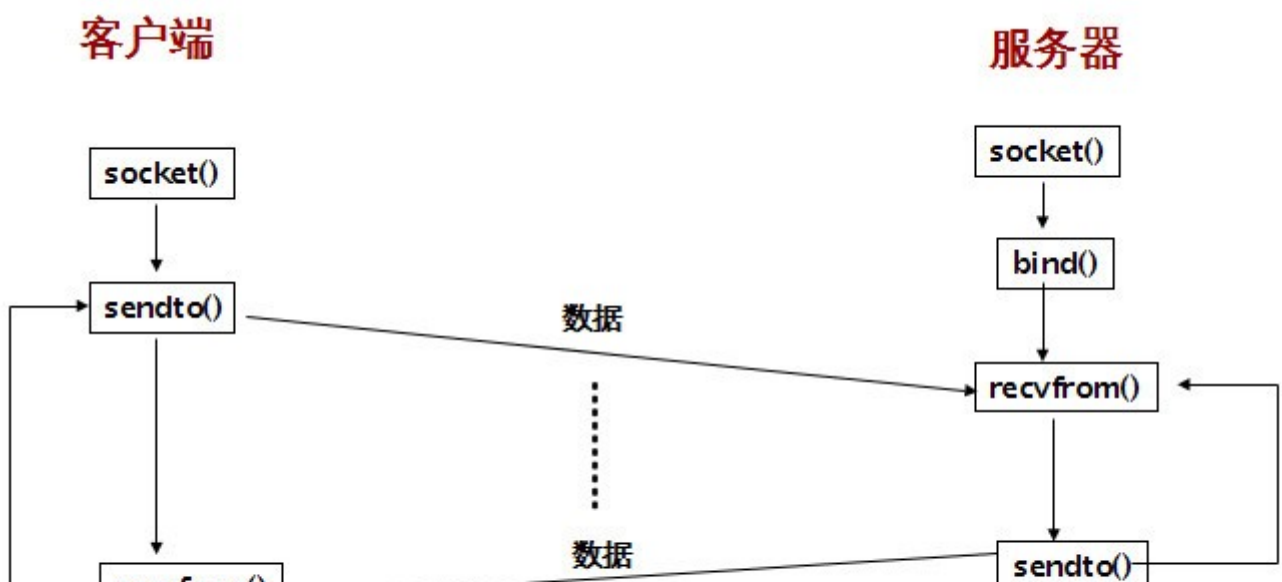
1、UDP绑定

```
1 int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
2
3
4 addr中端口为0，则使用临时端口号
```

注意：UDP是有可能出现接收缓冲区满，再接收数据时丢包。

```
1 1.1.改变接收缓冲区大小。
2     int n = 220x1024;
3     setsockopt(sockfd, SOL_SOCKET, SO_RCVBUF, &n, sizeof(n));
4
5
6 1.2.服务器应用层设计流量控制，控制发送数据速度。
```

2、UDP通信流程





3、多播（组播）

组播组可以是永久的也可以是临时的。

组播组地址中，一部分由官方分配的，称为永久组播组。永久组播组保存不变的是她的IP地址，组中的成员构成可以发生变化。永久组播组中成员的数量都可以是任意的，甚至可以为零。

- 1 224.0.0.0~224.0.0.255 为预留的组播地址（永久组地址），地址224.0.0.0保留不做分配，其它
- 2 224.0.1.0~224.0.1.255 是公用组播地址，可以用于Internet；
- 3 224.0.2.0~238.255.255.255 为用户可用的组播地址（临时组地址），全网范围内有效；
- 4 239.0.0.0~239.255.255.255 为本地管理组播地址，仅在特定的本地范围内有效。

查看网卡编号：ip ad

```
1 struct ip_mreqn
2 {
3     struct in_addr  imr_multiaddr; // 组播组的IP地址。
4     struct in_addr  imr_address;   // 本地IP
5     int             imr_ifindex;   // 网卡名的索引号if_nametoindex("eth0");
6 };
```

3.1、server.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <sys/types.h>
4 #include <sys/socket.h>
5 #include <string.h>
6 #include <unistd.h>
7 #include <arpa/inet.h>
```

```

8 #include <net/if.h>
9
10
11 #define SERVER_PORT 8000
12 #define CLIENT_PORT 9000
13 #define MAXLINE 4096
14
15
16 #define GROUP "239.0.0.2"
17
18
19 int main(void)
20 {
21     int sockfd,i;
22     struct sockaddr_in serveraddr,clientaddr;
23     char buf[MAXLINE] = "yzmin\n";
24     char ipstr[INET_ADDRSTRLEN];
25     socklen_t clientlen;
26     ssize_t len;
27     //组播结构体
28     struct ip_mreqn group;
29     //构造用于UDP通信的套接字
30     sockfd = socket(AF_INET,SOCK_DGRAM,0);
31
32
33     bzero(&serveraddr,sizeof(serveraddr));
34     serveraddr.sin_family = AF_INET;
35     serveraddr.sin_addr.s_addr = htonl(INADDR_ANY);
36     serveraddr.sin_port = htons(SERVER_PORT);
37
38
39     bind(sockfd,(struct sockaddr *)&serveraddr,sizeof(serveraddr));
40
41
42     //设置组地址
43     inet_pton(AF_INET,GROUP,&group.imr_multiaddr);
44     //本地任意IP
45     inet_pton(AF_INET,"0.0.0.0",&group.imr_address);
46     //eth0 --> 编号 命令:ip ad
47     group.imr_ifindex = if_nametoindex("eth0");
48     //设置组播
49     setsockopt(sockfd,IPPROTO_IP,IP_MULTICAST_IF,&group,sizeof(group));
50
51

```

```

51
52 //构造 client 地址 IP+端口
53 bzero(&clientaddr, sizeof(clientaddr));
54 clientaddr.sin_family = AF_INET;
55 inet_pton(AF_INET, GROUP, &clientaddr.sin_addr.s_addr);
56 clientaddr.sin_port = htons(CLIENT_PORT);
57
58
59 while(1)
60 {
61     sendto(sockfd, buf, strlen(buf), 0, (struct sockaddr *)&clientaddr, sizeof(clienta
62     sleep(1);
63 }
64 close(sockfd);
65 return 0;
66 }

```

3.2、client.c

```

1 #include <netinet/in.h>
2 #include <stdio.h>
3 #include <sys/types.h>
4 #include <sys/socket.h>
5 #include <arpa/inet.h>
6 #include <string.h>
7 #include <stdlib.h>
8 #include <sys/stat.h>
9 #include <unistd.h>
10 #include <fcntl.h>
11 #include <net/if.h>
12
13
14 // #define SERVER_PORT 8000
15 #define MAXLINE 4096
16
17
18 #define CLIENT_PORT 9000
19 #define GROUP "239.0.0.2"
20

```

```

21
22 int main(int argc,char *argv[])
23 {
24     struct sockaddr_in serveraddr,localaddr;
25     int confd;
26     ssize_t len;
27     char buf[MAXLINE];
28     //组播结构体
29     struct ip_mreqn group;
30
31
32     //1.创建一个socket
33     confd = socket(AF_INET,SOCK_DGRAM,0);
34
35
36     //2、初始化本地端地址
37     bzero(&localaddr,sizeof(localaddr));
38     localaddr.sin_family = AF_INET;
39     inet_pton(AF_INET,"0.0.0.0",&localaddr.sin_addr.s_addr);
40     localaddr.sin_port = htons(CLIENT_PORT);
41
42
43     bind(confd,(struct sockaddr *)&localaddr,sizeof(localaddr));
44
45
46     //设置组地址
47     inet_pton(AF_INET,GROUP,&group.imr_multiaddr);
48     //本地任意IP
49     inet_pton(AF_INET,"0.0.0.0",&group.imr_address);
50     //eth0 -->编号
51     group.imr_ifindex = if_nametoindex("eth0");
52
53
54     //设置client加入多播组
55     setsockopt(confd,IPPROTO_IP,IP_ADD_MEMBERSHIP,&group,sizeof(group));
56
57
58     while(1)
59     {
60         len = recvfrom(confd,buf,sizeof(buf),0,NULL,0);
61         write(STDOUT_FILENO,buf,len);
62     }
63
64

```

```
65     close(confd);
66     return 0;
67 }
```

4、广播

主机之间一对所有的通讯模式，网络对其中每一台主机发出的信号都进行无条件复制并转发，所有主机都可以接收到所有信息（不管你是否需要），由于其不用路径选择，所以其网络成本可以很低廉。

4.1、server.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <sys/types.h>
4  #include <sys/socket.h>
5  #include <string.h>
6  #include <unistd.h>
7  #include <arpa/inet.h>
8  #include <net/if.h>
9
10
11 #define SERVER_PORT 8000
12 #define CLIENT_PORT 9000
13 #define MAXLINE 4096
14
15
16 #define BROADCAST_IP "192.168.152.255"
17
18
19 int main(void)
20 {
21     int sockfd,i;
22     struct sockaddr_in serveraddr,clientaddr;
23     char buf[MAXLINE] = "yzmin\n";
24     char ipstr[INET_ADDRSTRLEN];
25     socklen_t clientlen;
```

```

26     ssize_t len;
27     struct ip_mreqn group;
28     //构造用于UDP通信的套接字
29     sockfd = socket(AF_INET, SOCK_DGRAM, 0);
30
31
32     bzero(&serveraddr, sizeof(serveraddr));
33     serveraddr.sin_family = AF_INET;
34     serveraddr.sin_addr.s_addr = htonl(INADDR_ANY);
35     serveraddr.sin_port = htons(SERVER_PORT);
36     bind(sockfd, (struct sockaddr *)&serveraddr, sizeof(serveraddr));
37
38
39     int flag = 1; //开启广播
40     setsockopt(sockfd, SOL_SOCKET, SO_BROADCAST, &flag, sizeof(flag));
41
42
43     //构造 client 地址 IP+端口
44     bzero(&clientaddr, sizeof(clientaddr));
45     clientaddr.sin_family = AF_INET;
46     inet_pton(AF_INET, BROADCAST_IP, &clientaddr.sin_addr.s_addr);
47     clientaddr.sin_port = htons(CLIENT_PORT);
48
49
50     while(1)
51     {
52         //fgets(buf, sizeof(buf), stdin);
53         sendto(sockfd, buf, strlen(buf), 0, (struct sockaddr *)&clientaddr, sizeof(clientaddr));
54         sleep(1);
55     }
56     close(sockfd);
57     return 0;
58 }

```

4.2、client.c

```

1 #include <netinet/in.h>
2 #include <stdio.h>
3 #include <sys/types.h>

```

```
4 #include <sys/socket.h>
5 #include <arpa/inet.h>
6 #include <string.h>
7 #include <stdlib.h>
8 #include <sys/stat.h>
9 #include <unistd.h>
10 #include <fcntl.h>
11 #include <net/if.h>
12
13
14 // #define SERVER_PORT 8000
15 #define MAXLINE 4096
16
17
18 #define CLIENT_PORT 9000
19
20
21 int main(int argc, char *argv[])
22 {
23     struct sockaddr_in localaddr;
24     int confd;
25     ssize_t len;
26     char buf[MAXLINE];
27
28
29     // 1. 创建一个socket
30     confd = socket(AF_INET, SOCK_DGRAM, 0);
31
32
33     // 2. 初始化本地端地址
34     bzero(&localaddr, sizeof(localaddr));
35     localaddr.sin_family = AF_INET;
36     inet_pton(AF_INET, "0.0.0.0", &localaddr.sin_addr.s_addr);
37     localaddr.sin_port = htons(CLIENT_PORT);
38     bind(confd, (struct sockaddr *)&localaddr, sizeof(localaddr));
39
40
41     while(1)
42     {
43         len = recvfrom(confd, buf, sizeof(buf), 0, NULL, 0);
44         write(STDOUT_FILENO, buf, len);
45     }
46
47
```



```
47
48     close(confd);
49     return 0;
50 }
```

socket IPC

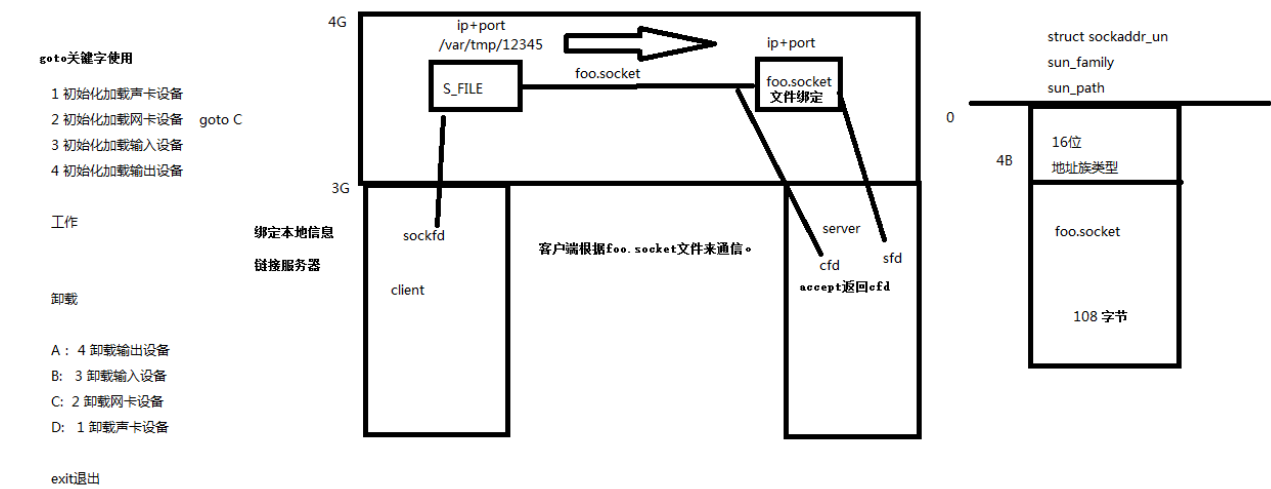
socket API原本是为网络通讯设计的，但后来在socket的框架上发展出一种IPC机制，就是UNIX Domain Socket。

不需要经过网络协议栈，不需要打包拆包、计算校验和、维护序号和应答等，只是将应用层数据从一个进程拷贝到另一个进程。

UNIX Domain Socket也提供面向流和面向数据包两种API接口，类似于TCP和UDP，但是面向消息的UNIX Domain Socket也是可靠的，消息既不会丢失也不会顺序错乱。

使用UNIX Domain Socket的过程和网络socket十分相似，也要先调用socket()创建一个socket文件描述符，address family指定为AF_UNIX，type可以选择SOCK_DGRAM或SOCK_STREAM，protocol参数仍然指定为0即可。

UNIX Domain Socket与网络socket编程最明显的不同在于地址格式不同，用结构体sockaddr_un表示，网络编程的socket地址是IP地址加端口号，而UNIX Domain Socket的地址是一个socket类型的文件在文件系统中的路径，这个socket文件由bind()调用创建，如果调用bind()时该文件已存在，则bind()错误返回。



1、将UNIX Domain socket绑定到一个地址

```
1 size = offsetof(struct sockaddr_un, sun_path) + strlen(un.sun_path);
2 #define offsetof(TYPE, MEMBER) ((int)&((TYPE *)0)->MEMBER)
```

2、 server.c

```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <stddef.h>
4 #include <sys/socket.h>
5 #include <sys/un.h>
6 #include <sys/types.h>
7 #include <sys/stat.h>
8 #include <unistd.h>
9 #include <errno.h>
10 #define QLEN 10
11
12
13 /*
14  * * Create a server endpoint of a connection.
15  * * Returns fd if all OK, <0 on error.
16  * */
17 int serv_listen(const char *name)
18 {
19     int fd, len, err, rval;
20     struct sockaddr_un un;
21     /* create a UNIX domain stream socket */
22     if ((fd = socket(AF_UNIX, SOCK_STREAM, 0)) < 0)
23         return(-1);
24     unlink(name); /* in case it already exists */
25     /* fill in socket address structure */
26     memset(&un, 0, sizeof(un));
27     un.sun_family = AF_UNIX;
28     strcpy(un.sun_path, name);
29     //偏移位置
30     len = offsetof(struct sockaddr_un, sun_path) + strlen(name);
31     /* bind the name to the descriptor */
```

```

32     if (bind(fd, (struct sockaddr *)&un, len) < 0) {
33         rval = -2;
34         goto errout;
35     }
36     if (listen(fd, QLEN) < 0) { /* tell kernel we're a server */
37         rval = -3;
38         goto errout;
39     }
40     return(fd);
41 errout:
42     err = errno;
43     close(fd);
44     errno = err;
45     return(rval);
46 }
47
48
49 int serv_accept(int listenfd, uid_t *uidptr)
50 {
51     int clifd, len, err, rval;
52     time_t staletime;
53     struct sockaddr_un un;
54     struct stat statbuf;
55     len = sizeof(un);
56     if ((clifd = accept(listenfd, (struct sockaddr *)&un, &len)) < 0)
57         return(-1); /* often errno=EINTR, if signal caught */
58     /* obtain the client's uid from its calling address */
59     len -= offsetof(struct sockaddr_un, sun_path); /* len of pathname */
60     un.sun_path[len] = 0; /* null terminate '\0' */
61     if (stat(un.sun_path, &statbuf) < 0) {
62         rval = -2;
63         goto errout;
64     }
65     if (S_ISSOCK(statbuf.st_mode) == 0) {
66         rval = -3; /* not a socket */
67         goto errout;
68     }
69     //文件所有者
70     if (uidptr != NULL)
71         *uidptr = statbuf.st_uid; /* return uid of caller */
72
73
74     unlink(un.sun_path); /* we're done with pathname now */
75     return(clifd);

```

```

75     return(cifd);
76 errout:
77     err = errno;
78     close(cifd);
79     errno = err;
80     return(rval);
81 }
82
83
84 int main(void)
85 {
86     int lfd, cfd, n, i;
87     uid_t cuid;
88     char buf[1024];
89     lfd = serv_listen("foo.socket");
90     if (lfd < 0) {
91         switch (lfd) {
92             case -3: perror("listen"); break;
93             case -2: perror("bind"); break;
94             case -1: perror("socket"); break;
95         }
96         exit(-1);
97     }
98     cfd = serv_accept(lfd, &cuid);
99     if (cfd < 0) {
100         switch (cfd) {
101             case -3: perror("not a socket"); break;
102             case -2: perror("a bad filename"); break;
103             case -1: perror("accept"); break;
104         }
105         exit(-1);
106     }
107     while (1) {
108 r_again:
109         n = read(cfd, buf, 1024);
110         if (n == -1) {
111             if (errno == EINTR)
112                 goto r_again;
113         }
114         else if (n == 0) {
115             printf("the other side has been closed.\n");
116             break;
117         }
118         for (i = 0; i < n; i++)

```

```

119         buf[i] = toupper(buf[i]);
120         write(cfd, buf, n);
121     }
122     close(cfd);
123     close(lfd);
124     return 0;
125 }

```

3、 client.c

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <stddef.h>
4 #include <sys/stat.h>
5 #include <fcntl.h>
6 #include <unistd.h>
7 #include <sys/socket.h>
8 #include <sys/un.h>
9 #include <errno.h>
10 #define CLI_PATH "/var/tmp/" /* +5 for pid = 14 chars */
11 /*
12  * * Create a client endpoint and connect to a server.
13  * * Returns fd if all OK, <0 on error.
14  * */
15 int cli_conn(const char *name)
16 {
17     int fd, len, err, rval;
18     struct sockaddr_un un;
19     /* create a UNIX domain stream socket */
20     if ((fd = socket(AF_UNIX, SOCK_STREAM, 0)) < 0)
21         return(-1);
22     /* fill socket address structure with our address */
23     memset(&un, 0, sizeof(un));
24     un.sun_family = AF_UNIX;
25     sprintf(un.sun_path, "%s%05d", CLI_PATH, getpid());
26     len = offsetof(struct sockaddr_un, sun_path) + strlen(un.sun_path);
27     unlink(un.sun_path); /* in case it already exists */
28     if (bind(fd, (struct sockaddr *)&un, len) < 0) {

```

```

29     rval = -2;
30     goto errout;
31 }
32 /* fill socket address structure with server's address */
33 memset(&un, 0, sizeof(un));
34 un.sun_family = AF_UNIX;
35 strcpy(un.sun_path, name); //foo.socket
36 len = offsetof(struct sockaddr_un, sun_path) + strlen(name);
37 if (connect(fd, (struct sockaddr *)&un, len) < 0) {
38     rval = -4;
39     goto errout;
40 }
41 return(fd);
42 errout:
43     err = errno;
44     close(fd);
45     errno = err;
46     return(rval);
47 }
48
49
50 int main(void)
51 {
52     int fd, n;
53     char buf[1024];
54     fd = cli_conn("foo.socket");
55     if (fd < 0) {
56         switch (fd) {
57             case -4: perror("connect"); break;
58             case -3: perror("listen"); break;
59             case -2: perror("bind"); break;
60             case -1: perror("socket"); break;
61         }
62         exit(-1);
63     }
64     while (fgets(buf, sizeof(buf), stdin) != NULL) {
65         write(fd, buf, strlen(buf));
66         n = read(fd, buf, sizeof(buf));
67         write(STDOUT_FILENO, buf, n);
68     }
69     close(fd);
70     return 0;
71 }

```

